

Project 2

AUTHOR
Grace Fischer

PUBLISHED
October 4, 2026

Setting Up

```
## Installing and calling library to prepare for assignment tasks
```

```
install.packages("tidyverse")  
install.packages("tidytuesdayR")
```

Part 1: Fun with Functions

Part 1A: Exponential Transformation

```
## Pasting function set up from assignment details  
library(tidyverse)  
library(tidytuesdayR)  
  
Exp <- function(x, k) {  
  ## Accounting for the constant term  
  result <- 1  
  ## Using i to make seq_len from Lecture 8 Notes to make  
  ## sure we have as i goes to k  
  for (i in seq_len(k)){  
    result <- result + x^i / factorial(i)  
  }  
  result  
}  
  
## Example output:  
cat("First example result from Exp(x = 1, k = 1):", Exp(x = 1,  
  k = 1), "\n") ## This is simply, 1 + 1, output should  
  be 2
```

First example result from Exp(x = 1, k = 1): 2

```
cat("Second example result from Exp(x = 4, k = 2):", Exp(x =
      4, k = 2), "\n") ## Output should be 13
```

Second example result from Exp(x = 4, k = 2): 13

Part 1B: Sample Mean and Standard Deviation

```
sample_mean <- function(x) {
  ## Set length of the vector x to be N for the sample
  N <- length(x)
  ## Instantiating total variable
  total <- 0
  ## Using seq_along to loop over a vector this time, like
  ## in Lecture 8 notes
  for (i in seq_along(x)){
    ## Updating total variable
    total <- total + x[i]
  }
  total / N
}

sample_sd <- function(x) {
  ## Set length of the vector x to be N for the sample
  N <- length(x)
  ## Instantiating ss variable
  ss <- 0
  ## Creating xbar variable with sample_mean function we
  ## created
  xbar <- sample_mean(x)
  ## Using seq_along to loop over a vector this time, like
  ## in Lecture 8 notes
  for (i in seq_along(x)){
    ## Updating ss variable using first part of formula
    ss <- ss + (x[i] - xbar)^2
  }
  sqrt(ss/ (N - 1))
}

## Example outputs
## I based expected outputs based off hand calculations, so I
## rounded (hence the tildes)
```

```
x1 <- c(1,2,3,4,5,6,7,8,9,10)
cat("From x1 <- c(1,2,3,4,5,6,7,8,9,10), we see a sample mean
of", sample_mean(x1), "and a sample sd of",
sample_sd(x1), "\n")## Expected output: 5.5, Expected
output: ~ 3.02
```

From `x1 <- c(1,2,3,4,5,6,7,8,9,10)`, we see a sample mean of 5.5 and a sample sd of 3.02765

```
x2 <- c(2,4,6,8,8,6,4,2)
cat("From x2 <- c(2,4,6,8,8,6,4,2), we see a sample mean of",
sample_mean(x2), "and a sample sd of", sample_sd(x2),
"\n")## Expected output: 5, Expected output: ~ 2.4
```

From `x2 <- c(2,4,6,8,8,6,4,2)`, we see a sample mean of 5 and a sample sd of 2.390457

Part 1C: Confidence Intervals

```
## Pasted from assignment directions
calculate_CI <- function(x, conf = 0.95) {
  ## Creating variables based on previous parts of the
  ## problem, and standard error calculations of mean
  N <- length(x)
  xbar <- sample_mean(x)
  se <- sample_sd(x) / sqrt(N)

  alpha <- 1 - conf
  degrees_freedom <- N - 1
  t_score <- qt(p = alpha / 2, df = degrees_freedom,
lower.tail = FALSE)

  margin <- t_score * se
  c(lower_bound = xbar - margin, upper_bound = xbar +
margin)
}

## Example outputs
## Using x1, and x2 from earlier where
## x1 <- c(1,2,3,4,5,6,7,8,9,10)
## x2 <- c(2,4,6,8,8,6,4,2)

##calculate_CI(x1, conf = 0.95)
```

```
cat("Using our function we created to find confidence interval
    using x1 from earlier and conf = .95, we
    see", calculate_CI(x1, conf = 0.95), "\n")
```

Using our function we created to find confidence interval using x1 from earlier and conf = .95, we see 3.334149 7.665851

```
## Checking against R function
dat1 <- data.frame(x = x1)
fit1 <- lm(x ~ 1, dat1)
##confint(fit1, level = 0.95)

cat("Using the R function to check out function, we
    see", confint(fit1, level = 0.95), "\n")
```

Using the R function to check out function, we see 3.334149 7.665851

```
##calculate_CI(x2, conf = 0.98)

cat("Using our function we created to find confidence interval
    using x2 from ealier and conf = .98, we see",
    calculate_CI(x2, conf = 0.98), "\n")
```

Using our function we created to find confidence interval using x2 from ealier and conf = .98, we see 2.466268 7.533732

```
## Checking against R function
dat2 <- data.frame(x = x2)
fit2 <- lm(x ~ 1, dat2)
##confint(fit2, level = 0.98)

cat("Using the R function to check out function, we see",
    confint(fit2, level = 0.98), "\n")
```

Using the R function to check out function, we see 2.466268 7.533732

Part 2: Wrangling Data

Setting Up Data for Part 2

```
## Copy and pasted from website instructions
```

```
tuesdata <- tidyuesdayR::tt_load("2020-01-07")
rainfall <- tuesdata$rainfall
temperature <- tuesdata$temperature
```

```
## Copy and pasted from website instructions
```

```
library(here)
```

```
## Added directory line for additional repo
```

```
if (!dir.exists(here("data"))) {
  dir.create(here("data"))
}
```

```
if (!file.exists(here("data", "tuesdata_rainfall.RDS"))) {
  tuesdata <- tidyuesdayR::tt_load("2020-01-07")
  rainfall <- tuesdata$rainfall
  temperature <- tuesdata$temperature
```

```
# save the files to RDS objects
```

```
saveRDS(tuesdata$rainfall, file = here("data",
  "tuesdata_rainfall.RDS"))
saveRDS(tuesdata$temperature, file = here("data",
  "tuesdata_temperature.RDS"))
```

```
}
```

```
## Copy and pasted from website instructions
```

```
rainfall <- readRDS(here("data", "tuesdata_rainfall.RDS"))
temperature <- readRDS(here("data",
  "tuesdata_temperature.RDS"))
```

```
library(tidyverse)
```

```
glimpse(rainfall)
```

```
Rows: 179,273
```

```
Columns: 11
```

```
$ station_code <chr> "009151", "009151", "009151", "009151",
"009151", "009151..."
```

```
$ city_name <chr> "Perth", "Perth", "Perth", "Perth",
"Perth", "Perth", "Pe..."
```

```
$ year <dbl> 1967, 1967, 1967, 1967, 1967, 1967, 1967, 1967,
```

```

1967, 1967, 196...
$ month      <chr> "01", "01", "01", "01", "01", "01", "01",
"01", "01", "01...
$ day        <chr> "01", "02", "03", "04", "05", "06", "07",
"08", "09", "10...
$ rainfall   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
NA, NA, NA, NA, N...
$ period     <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
NA, NA, NA, NA, N...
$ quality    <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,
NA, NA, NA, NA, N...
$ lat        <dbl> -31.96, -31.96, -31.96, -31.96, -31.96,
-31.96, -31.96, -...
$ long       <dbl> 115.79, 115.79, 115.79, 115.79, 115.79,
115.79, 115.79, 1...
$ station_name <chr> "Subiaco Wastewater Treatment Plant",
"Subiaco Wastewater..."

```

```
glimpse(temperature)
```

```

Rows: 528,278
Columns: 5
$ city_name  <chr> "PERTH", "PERTH", "PERTH", "PERTH",
"PERTH", "PERTH", "PER...
$ date       <date> 1910-01-01, 1910-01-02, 1910-01-03, 1910-
01-04, 1910-01-0...
$ temperature <dbl> 26.7, 27.0, 27.5, 24.0, 24.8, 24.4, 25.3,
28.0, 32.6, 35.9...
$ temp_type  <chr> "max", "max", "max", "max", "max", "max",
"max", "max", "m...
$ site_name  <chr> "PERTH AIRPORT", "PERTH AIRPORT", "PERTH
AIRPORT", "PERTH ..."

```

Task Portion

```
## Check dim in rainfall
dim(rainfall)
```

```
[1] 179273    11
```

```
## Creating data frame
df <- rainfall %>%
```

```
## Dropping NA's from the columns (source below)
drop_na() %>%

## remove = false (source below)
unite(col = "date", year:day, sep = "-",
      remove = FALSE) %>%

## ymd from lubridate (source below)
mutate(date = ymd(date)) %>%
select(-month, -day) %>%

## using str fx from assignment hint (Source below)
mutate(city_name = str_to_upper(city_name)) %>%
inner_join(temperature, by = c("city_name", "date"),
           relationship = "many-to-many")

## Checking new dimensions of df to make sure edits were made
dim(df)
```

```
[1] 83964    13
```

Sources I used to help me complete this section:

https://tidyr.tidyverse.org/reference/drop_na.html

<https://tidyr.tidyverse.org/reference/unite.html>

<https://lubridate.tidyverse.org/reference/ymd.html>

<https://stringr.tidyverse.org/reference/case.html>

Part 3: Data Visualization

```
install.packages("paletteer")
install.packages("ggthemes")
```

Sources I used to help me complete this:

<https://r-charts.com/color-palettes/>

Part 3A: Plotting temperature data over time

```

## library for unique color values
library(paletteer)
library(ggthemes)

## Editing data frame
df %>%
  filter(year >= 2014) %>%

## Making plot w aes, setting x and y respectively
ggplot(aes(x = date, y = temperature, color = temp_type))
  +
  geom_line(alpha = 0.9) +
  facet_grid(city_name ~ .) +

## Scale taken from paletteer package
scale_colour_manual(

  ## Hex Codes source below
  values = paletteer_c("ggthemes::Classic Orange-Blue",
    30)[c(8, 23)],
  ## If I set it normally, I would do it this way:
  ## values = c("max" = "orange", "min" = "blue"),
  labels = c("max" = "Daily max", "min" = "Daily min")
) +

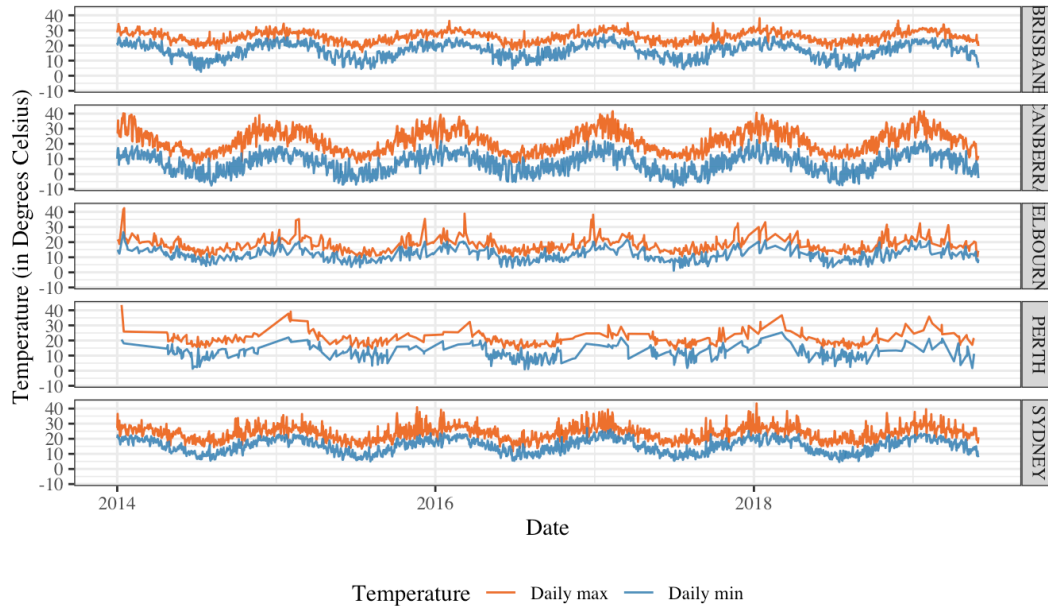
## Setting x and y labels, as well as titles and subtitles
labs(
  x = "Date",
  y = "Temperature (in Degrees Celsius)",
  color = "Temperature",
  title = "Daily Maximum and Minimum Temperatures in
    Australian Cities, 2014–2019",
  subtitle = "All cities follow a clear summer–winter
    cycle; Brisbane is warmest on average,\nwhile
    Canberra has the widest gap between highs and lows",
  caption = "Data: Australian Bureau of Meteorology"
) +
theme_bw(base_family = "Times") +

## Source below
theme(
  plot.title = element_text(face = "bold", size = 14),
  legend.position = "bottom"
)

```

Daily Maximum and Minimum Temperatures in Australian Cities, 2014–2019

All cities follow a clear summer–winter cycle; Brisbane is warmest on average, while Canberra has the widest gap between highs and lows



Data: Australian Bureau of Meteorology

Sources I used to help me complete this section:

<https://ggplot2.tidyverse.org/reference/theme.html>

<https://r-charts.com/color-palettes/>

Part 3B: Plotting rainfall over time

```
## Creating function
plot_rainfall <- function(city_name, year) {
  ## Check inputs so that exactly one city and one year were
  ## entered
  if (length(city_name) != 1L || length(year) != 1L) {
    stop("Please enter one city and one year at a time.",
         call. = FALSE)
  }

  ## Checks input so that a city was entered as text
  if (!is.character(city_name)) {
    stop("`city_name` must be text, such as `PERTH`.",
         call. = FALSE)
  }

  ## Checks input so that year was entered as number
```

```
if (!is.numeric(year)) {
  stop("`year` must be a number, such as 2000.", call. =
    FALSE)
}

## Rename input to not mess with df
city_input <- str_to_upper(city_name)
year_input <- year

## Check to see that the input exists in the data (sources
  below)
if (!(city_input %in% df$city_name)) {
  stop("", city_name, "' is not in the data. Choose
    from: ",
    paste(unique(df$city_name), collapse = ", "),
    call. = FALSE)
}

## Filtering for one row per rainfall measurement
new_df <- df %>%
  filter(city_name == city_input, year == year_input,
    temp_type == "max")

if (nrow(new_df) == 0) {
  ## Checking the years from rows where the city
    matches, like boolean conditions
  years_available <- unique(df$year[df$city_name ==
    city_input])
  stop("No data for ", city_input, " in ", year_input,
    ". Years available: ",
    min(years_available), " to ",
    max(years_available),
    " (some years in between may be missing).", call.
    = FALSE)
}

## Checking for number of days with no rain
days_no_rain <- sum(new_df$rainfall == 0)
## Checking for rainy days
rainy <- new_df %>% filter(rainfall > 0)

## Creating the plot
rainy %>%
  ## Plotting log transformation for better visibility
  ggplot(aes(x = log(rainfall))) +
```

```

## Setting Color for graph aesthetics / Made bins 20
because I got a message to select better bin number
then 30 which is preset value
geom_histogram(bins = 20, fill = "deepskyblue2", color
= "darkslategrey") +
labs(
  x = "log(Daily Rainfall in mm)",
  y = "Number of days",
  title = paste("Distribution of Daily Rainfall in",
city_input, "in", year_input),
  subtitle = paste0(nrow(rainy), " rainy days
(median ",
                                round(median(rainy$rainfall),
1), " mm); ",
                                days_no_rain, " dry days not
shown on the log scale"),
  caption = "Data: Australian Bureau of Meteorology"
) +
theme_minimal(base_family = "Georgia")+

theme(
  plot.title = element_text(face = "bold", size = 14)
)
}

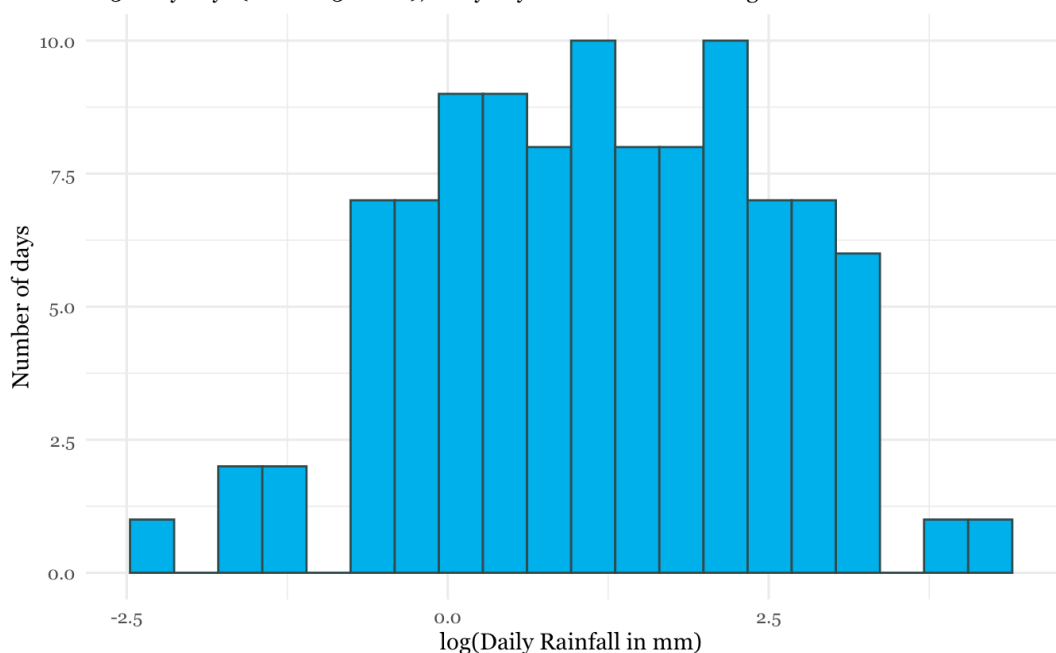
```

See output in following Cell

```
plot_rainfall("Perth", 2000)
```

Distribution of Daily Rainfall in PERTH in 2000

103 rainy days (median 3.2 mm); 0 dry days not shown on the log scale



Data: Australian Bureau of Meteorology

Error where city is not in data (I googled popular cities and chose the first one not included in data)

```
plot_rainfall("ADELAIDE", 2000)
```

Error:

! 'ADELAIDE' is not in the data. Choose from: PERTH, BRISBANE, SYDNEY, CANBERRA, MELBOURNE

Error where city exists, but year does not

```
plot_rainfall("CANBERRA", 1850)
```

Error:

! No data for CANBERRA in 1850. Years available: 2008 to 2019 (some years in between may be missing).

Error where too many inputs

```
plot_rainfall("PERTH", c(2000, 2001))
```

Error:

! Please enter one city and one year at a time.

Sources I used to help me complete this:

<https://www.geeksforgeeks.org/r-language/collapse-function-in-r/>

<https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/unique>

Function Design Explanation

I designed `plot_rainfall()` to take two arguments, "city_name" and "year". It has one return which is histogram of daily rainfall on a log scale for that combination. I decided to use the log scale for the histogram to have better visibility/ scale. The repeated calling allows the user to engage in data exploration, without needing to copy and paste the code.

Following the error-checking approach from class, the function checks the user's inputs in different stages prior to doing any of the actual work or graphing. First, it confirms that exactly one city (as text) and one year (as a number) were entered. It then checks that the city exists in the data. Finally, the last error checking measure is that the city has measurements for the requested year. My minor in computer science from undergrad taught me that a good error message should not just tell users that something went wrong, but also show them what they can do instead, so they do not keep making the same mistake. This has remained true through all my CS courses, in various programming languages, so I wanted to try to challenge myself and implement that here in R, since that is not something I have done before. Therefore, each `stop()` message explains how to fix the input, such as listing the valid cities or the range of years available for that city. I also convert the city name to upper case, so "perth" and "Perth" work as well as "PERTH".

Inside the function, I copy the inputs to new names, because when we filter, it would compare the column to itself and keep every row. I also keep only the rows where `temp_type == max`, since the join in Part 2 matched each rainfall measurement to both a maximum and a minimum temperature, which would otherwise count every day twice.

Finally, when it comes to plotting, I plotted `log(rainfall)` because the daily rainfall has a very positive skew. The log transformation just makes the shape of the distribution more visible. Because `log(0)` is undefined, I exclude days with no rain from the histogram and report how many there were in the subtitle of the plot. The title and subtitle are built with `paste()`, so they update automatically for each city and year.

Part 4: Apply functions and plot

Part 4A: Tasks

```
## Create new df
rain_df <- df %>%
  ## Filter for 2014 and onwards, additionally, make one row
  ## per rainfall measure
  filter(year >= 2014, temp_type == "max") %>%

  ## Create one group per city and year
  group_by(city_name, year) %>%

  ## Summarize using the functions we created
  summarize(
    mean_rainfall = sample_mean(rainfall),
    sd_rainfall   = sample_sd(rainfall),
    lower_bound   = calculate_CI(rainfall, conf = 0.95)
    ["lower_bound"],
    upper_bound   = calculate_CI(rainfall, conf = 0.95)
    ["upper_bound"],
  ) %>%
  ## Do this to avoid error later
  ungroup()

rain_df
```

```
# A tibble: 30 × 6
  city_name year mean_rainfall sd_rainfall lower_bound
upper_bound
  <chr>      <dbl>          <dbl>          <dbl>          <dbl>
<dbl>
1 BRISBANE  2014              3.84           12.3           2.64
5.05
2 BRISBANE  2015              6.17           22.8           3.85
8.50
3 BRISBANE  2016              2.09            8.61           1.20
2.98
4 BRISBANE  2017              2.72           10.2           1.66
3.78
5 BRISBANE  2018              2.47            9.63           1.45
3.48
6 BRISBANE  2019              2.10            6.36           1.07
```

```

3.12
  7 CANBERRA    2014          1.57          4.84          1.07
2.07
  8 CANBERRA    2015          1.51          5.16          0.977
2.05
  9 CANBERRA    2016          2.15          6.28          1.51
2.80
 10 CANBERRA    2017          1.34          4.12          0.913
1.76
# i 20 more rows

```

Part 4B: Tasks

```

## Use ggplot and new df to make graph
rain_df %>%

  ## Using aes, setting x and y respectively
  ggplot(aes(x = year, y = mean_rainfall)) +

  ## Error bars for the 95% confidence interval, sources
  ## below
  geom_errorbar(aes(ymin = lower_bound, ymax = upper_bound),
                width = 0.2, color = "darkblue") +

  ## Line and points for the means
  geom_line(color = "darkblue") +
  geom_point(color = "darkblue", size = 2) +

  ## Source below
  facet_wrap(~ city_name, nrow = 1) +

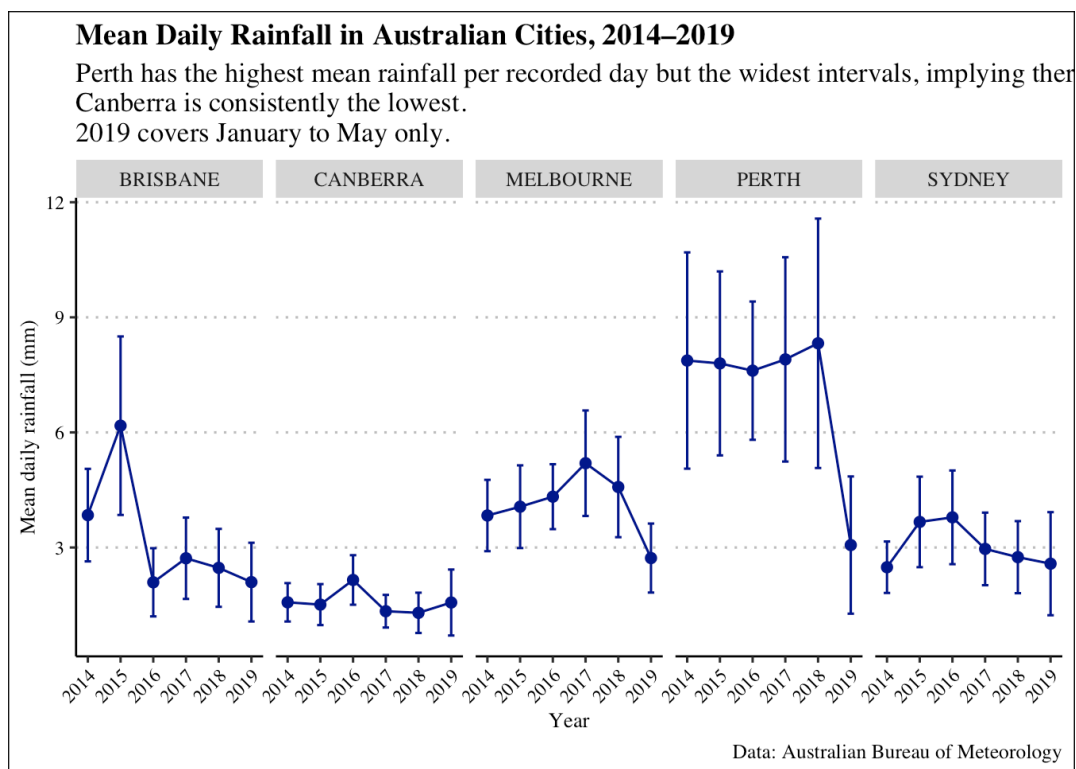
  ## ggplot2 function which controls numeric x axis
  ## (inspired by Lecture 6 notes, source below)
  scale_x_continuous(breaks = 2014:2019) +

  labs(
    title = "Mean Daily Rainfall in Australian Cities,
    2014–2019",
    subtitle = "Perth has the highest mean rainfall per
    recorded day but the widest intervals, implying there
    are fewer measurements.\nCanberra is consistently the
    lowest.\n2019 covers January to May only.",
    caption = "Data: Australian Bureau of Meteorology",
    x = "Year",
    y = "Mean daily rainfall (mm)"
  ) +

  theme_clean(base_family = "Times") +

```

```
theme(
  plot.title = element_text(face = "bold", size = 14),
  axis.text.x = element_text(angle = 45, hjust = 1)
)
```



Sources that helped me complete this (from instructions included):

<https://r-graphics.org/RECIPE-ANNOTATE-ERROR-BAR.html>

https://ggplot2.tidyverse.org/reference/geom_linerange.html

https://ggplot2.tidyverse.org/reference/facet_wrap.html

https://ggplot2.tidyverse.org/reference/scale_continuous.html